

Versuchsprotokoll Versuch 5 - VHDL / FPGA

Praktikum: Digitaltechnik
Versuch: 5 - VHDL / FPGA
Name: Precious Adaugo Nnamaka
Betreuer: Prof. Dr.-Ing. Peter Zipf

1 Ziel des Versuchs

Ziel des fünften Versuchs war der Übergang von einer grafisch entwickelten Digitalschaltung zu einer Hardwarebeschreibung in VHDL und deren Umsetzung auf einem FPGA. Dazu wurde zunächst ein 4-Bit-zu-7-Segment-Decoder in Digital entworfen und geprüft. Anschließend wurde die Schaltung als VHDL beschrieben, in Vivado simuliert, synthetisiert und auf ein Digilent PYNQ-Z1-Board mit dem FPGA XC7Z020 übertragen.

Als zweiter Teil wurde ein synchroner Notengenerator betrachtet. Dieser erzeugt aus einem Eingangswert einen sechs Bit breiten Notenwert und zeigt damit den Unterschied zwischen rein kombinatorischen und zustandsbehafteten synchronen Schaltungen.

2 Verwendete Werkzeuge und Dateien

Werkzeug / Datei | Verwendung
Digital | Grafischer Entwurf und Simulation der Logikschaltungen
trial.dig | 4-Bit-zu-7-Segment-Decoder in Digital
trial.vhdl | Aus Digital exportierte VHDL-Beschreibung
Vivado | Simulation, Synthese, Implementation und Bitstream-Erzeugung
Testbench | Automatische Überprüfung der Eingangskombinationen
XDC-Datei | Zuordnung der Signale zu den FPGA-Pins
PYNQ-Z1 / XC7Z020 | Zielhardware für die FPGA-Umsetzung
Notengenerator.dig | Synchroner 6-Bit-Notengenerator
Audio-Projekt | Einbindung des Notengenerators in die Audioausgabe

3 Theoretische Grundlagen

VHDL beschreibt keine nacheinander ausgeführte Software, sondern digitale Hardware. Die Beschreibung legt fest, welche logischen Verknüpfungen, Register und Signalfade im FPGA realisiert werden. Eine VHDL-Datei besteht typischerweise aus einer Entity und einer Architecture. Die Entity beschreibt die Ein- und Ausgänge einer Schaltung. Die Architecture beschreibt das Verhalten oder die Struktur der Schaltung.

Wichtige VHDL-Bestandteile sind:

Bestandteil | Bedeutung
library IEEE / STD_LOGIC_1164 | Einbindung digitaler Logikdatentypen
entity | Definition der Ein- und Ausgänge
architecture | Beschreibung der Schaltungslogik
STD_LOGIC | Datentyp für einzelne digitale Signale
STD_LOGIC_VECTOR | Datentyp für mehrbitige Signale und Busse
Testbench | Simulation der Schaltung ohne reale Hardware
XDC-Datei | Verbindung zwischen VHDL-Signalen und FPGA-Pins

Ein FPGA arbeitet parallel. Mehrere Signalzuweisungen werden daher nicht nacheinander wie in einem normalen Programm ausgeführt, sondern als gleichzeitig vorhandene Hardwarestrukturen umgesetzt.

4 4-Bit-zu-7-Segment-Decoder

4.1 Entwurf in Digital

Der 7-Segment-Decoder besitzt vier Eingänge X0 bis X3. Diese vier Eingänge stellen eine Binärzahl dar. Die sieben Ausgänge a bis g steuern die einzelnen Segmente der Anzeige. Für jede Dezimalziffer von 0 bis 9 muss eine passende Kombination der Segmente aktiviert werden.

Die Schaltung besitzt keinen Speicher und keinen Takt. Die Ausgänge hängen ausschließlich von der aktuell anliegenden Eingangskombination ab. Der Decoder ist daher ein Schaltnetz beziehungsweise eine kombinatorische Schaltung.

In Digital wurde die Schaltung mit Schaltern an den Eingängen und einer Sieben-Segment-Anzeige an den Ausgängen geprüft. Für die Dezimalziffern 0 bis 9 wurde kontrolliert, ob die richtige Anzeige erscheint.

Überprüfung:

Binärwert | Dezimalziffer | Ergebnis

0000 | 0 | korrekt

0001 | 1 | korrekt

0010 | 2 | korrekt

0011 | 3 | korrekt

0100 | 4 | korrekt

0101 | 5 | korrekt

0110 | 6 | korrekt

0111 | 7 | korrekt

1000 | 8 | korrekt

1001 | 9 | korrekt

Die Simulation in Digital bestätigte die korrekte Funktion des Decoders. Der gespeicherte Entwurf trial.dig dokumentiert die grafische Schaltung.

4.2 Export nach VHDL

Nach der erfolgreichen Simulation wurde die Schaltung aus Digital nach VHDL exportiert. Die erzeugte Datei trial.vhdl enthält die logischen Gleichungen für die Segmentausgänge. Ein Beispiel aus der exportierten Beschreibung ist:

```
c <= X0 OR NOT X1 OR X2
```

Diese Zuweisung zeigt, dass VHDL die boolesche Funktion des Ausgangs c direkt beschreibt. Entsprechend werden auch die anderen Segmentausgänge a, b, d, e, f und g durch logische Verknüpfungen erzeugt.

5 Umsetzung in Vivado

Die Arbeit mit Vivado war ein zentraler Bestandteil des Versuchs. Im Unterschied zur grafischen Simulation in Digital mussten mehrere Dateien korrekt zusammenpassen: die VHDL-Quelle, die Testbench, die XDC-Pinbelegung und das gewählte FPGA-Zielgerät.

Der rekonstruierte Arbeitsablauf war:

1. Vivado öffnen und ein neues RTL-Projekt anlegen.
2. VHDL als Zielsprache auswählen.
3. Das Board PYNQ-Z1 beziehungsweise den FPGA XC7Z020 als Zielhardware auswählen.
4. Die VHDL-Designquelle hinzufügen.
5. Eine Testbench hinzufügen, die verschiedene Eingangskombinationen vorgibt.
6. Die XDC-Datei einbinden und die Eingänge sowie Segmentausgänge den realen Pins zuordnen.
7. Die Behavioral Simulation starten und die Ausgangssignale prüfen.
8. Synthese ausführen.
9. Implementation ausführen.
10. Bitstream erzeugen.
11. Das PYNQ-Z1 über USB verbinden.
12. Im Hardware Manager das Zielgerät verbinden und programmieren.
13. Die Funktion auf der realen Sieben-Segment-Anzeige überprüfen.

Besonders wichtig war, dass die Portnamen in der VHDL-Entity mit den Namen in der XDC-Datei übereinstimmen. Bereits kleine Abweichungen bei Signalnamen oder Pinzuordnungen konnten zu Fehlern bei Simulation, Synthese oder Programmierung führen.

6 Testbench und Pinbelegung

Die Testbench legte nacheinander verschiedene Binärwerte an die Eingänge an. Zwischen den Testwerten wurde jeweils eine Wartezeit verwendet, zum Beispiel 100 ns. Dadurch konnte in der Simulation überprüft werden, ob die Segmentausgänge für jede Eingangskombination korrekt reagieren.

Die XDC-Datei ordnete die Eingangssignale den Tastern beziehungsweise Schaltern des Boards zu. Die Ausgänge a bis g wurden den Pins der Sieben-Segment-Anzeige beziehungsweise der Pmod-Schnittstelle zugeordnet. Außerdem musste der passende elektrische Standard, zum Beispiel LVCMOS33, gesetzt werden.

7 Durchführung und Ergebnisse

Nach erfolgreicher Simulation wurden Synthese und Implementation durchgeführt. Anschließend wurde der Bitstream erzeugt und auf das PYNQ-Z1 übertragen. Das Board wurde im Hardware Manager erkannt und konnte programmiert werden.

Durch Betätigen der Eingangstaster beziehungsweise durch Anlegen der Eingangskombinationen änderte sich der Binärwert. Die zugehörige Dezimalziffer wurde auf der angeschlossenen Sieben-Segment-Anzeige dargestellt. Damit wurde die vorher in Digital und in der Testbench geprüfte Funktion auch auf realer Hardware bestätigt.

Die vollständige Entwurfskette wurde damit erfolgreich durchlaufen:

Wahrheitstabelle -> Gatternetz -> VHDL -> Simulation -> Synthese -> Implementation -> Bitstream -> FPGA-Test

8 Notengenerator

Im zweiten Teil wurde der Entwurf Notengenerator.dig betrachtet. Dieser besitzt einen vier Bit breiten Eingang btn_i und einen sechs Bit breiten Ausgang note_o. Der Eingangswert wird über kombinatorische Auswahlpfade verarbeitet. Zusätzlich enthält die Schaltung ein Register, sodass ein Zustand gespeichert werden kann.

Die wichtigsten Bestandteile des Notengenerators waren:

Baustein | Funktion

btn_i[3:0] | Auswahl beziehungsweise Steuerung der Note

Multiplexer | Auswahl zwischen Eingangswert, Rückführung und Resetpfad

Addierer | Bildung des nächsten Notenwertes

Register | Speicherung von note_o am Takt

reset_i | synchrones Zurücksetzen auf 0

note_o[5:0] | Notencode für die nachfolgende Tonerzeugung

Im Gegensatz zum 7-Segment-Decoder handelt es sich beim Notengenerator nicht um ein reines Schaltnetz. Durch das Register ist die Schaltung zustandsbehaftet und synchron. Der Ausgang note_o hängt daher nicht nur vom aktuellen Eingang ab, sondern auch vom gespeicherten Zustand.

9 Einbindung in die Audioausgabe

Der Notengenerator wurde in ein Audio-Projekt eingebunden. Dieses enthielt unter anderem folgende Module:

Modul | Aufgabe

Clock_Divider | Erzeugung langsamerer Arbeits- und Notentakte aus dem FPGA-Takt

Reset_module | Verwaltung des Reset-Signals

vol_ctrl | Steuerung der Lautstärke

PWM_Modulator | Erzeugung des digitalen Audiosignals

Audio_Sequence_Player | Abspielen der Notenfolge

Aus dem Notencode wurde eine passende Tonfrequenz abgeleitet. Der PWM-Modulator erzeugte daraus ein digitales Audiosignal, das anschließend über die Audio-Pins des Boards ausgegeben wurde.

10 Auswertung

Der Versuch verdeutlichte den Unterschied zwischen grafischer Logiksimulation und realer FPGA-Umsetzung. In Digital konnte die Schaltung anschaulich aufgebaut und direkt getestet werden. In Vivado mussten dagegen VHDL-Code, Testbench, Pinbelegung und Zielhardware korrekt zusammenarbeiten.

Der 7-Segment-Decoder zeigte eine rein kombinatorische Funktion. Eine Änderung der Eingänge wirkte sich direkt auf die Ausgänge aus, ohne dass ein Takt oder Speicher beteiligt war. Die erfolgreiche Hardwareprüfung bestätigte, dass die aus Digital exportierte Logik korrekt auf dem FPGA umgesetzt wurde.

Der Notengenerator zeigte dagegen eine synchrone, zustandsbehaftete Schaltung. Durch das Register blieb ein Notenwert gespeichert und wurde nur taktabhängig verändert. Dadurch kann der Kern des Notengenerators als Moore-ähnlicher Automat betrachtet werden, da der Ausgang aus dem gespeicherten Zustand hervorgeht.

11 Fehlerbetrachtung

Mögliche Fehlerquellen bei diesem Versuch waren:

1. Falsche oder abweichende Portnamen zwischen VHDL-Datei und XDC-Datei.
2. Nicht korrekt zugeordnete FPGA-Pins.
3. Fehlender oder falscher elektrischer Standard in der XDC-Datei.
4. Fehler in der Testbench, wodurch falsche Eingangskombinationen geprüft werden.
5. Unterschiedliche Signalreihenfolge zwischen Digital-Export und Vivado-Projekt.
6. Nicht erfolgreich ausgeführte Synthese oder Implementation.
7. Probleme beim Verbinden des PYNQ-Z1 im Hardware Manager.
8. Verwechslung zwischen kombinatorischer Logik und getakteter Registerlogik.

12 Fazit

Der Versuch zeigte den vollständigen Ablauf von einer digitalen Schaltungsidee bis zur Umsetzung auf einem FPGA. Der 4-Bit-zu-7-Segment-Decoder wurde zunächst in Digital geprüft, anschließend nach VHDL exportiert, in Vivado simuliert und schließlich auf dem PYNQ-Z1 getestet. Die Hardwareausgabe bestätigte die korrekte Funktion der kombinatorischen Schaltung.

Der Notengenerator ergänzte den Versuch um eine synchrone Schaltung mit gespeichertem Zustand. Dadurch wurde deutlich, dass VHDL sowohl reine Schaltnetze als auch getaktete Schaltwerke beschreiben kann. Insgesamt machte der Versuch sichtbar, dass bei FPGA-Projekten nicht nur die Logik selbst, sondern auch Testbench, Constraints, Synthese, Implementation und Hardwareprogrammierung zuverlässig zusammenpassen müssen.